

블록 디퓨전: 자기회귀와 디퓨전 언어모델 사이를 보간하기

(Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models — 한국어 PDF판)

Marianne Arriola, Aaron Kerem Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subha

한국어 번역: Compendium (<https://yoon-gu.github.io>)

Contents

0.1	초록	2
0.2	1. 서론	2
0.3	2. 배경: 언어모델링 패러다임	3
0.3.1	2.1 자기회귀 모델	3
0.3.2	2.2 이산 잡음제거 디퓨전 확률 모델 (D3PM)	3
0.4	3. 블록 디퓨전 언어모델링	4
0.4.1	3.1 블록 디퓨전 분포와 모델 아키텍처	4
0.4.2	3.2 효율적인 학습 및 샘플링 알고리즘	5
0.5	4. 디퓨전과 AR 모델 사이의 우도 갭 이해	5
0.5.1	4.1 마스킹 BD3-LM	6
0.5.2	4.2 사례 연구: 단일 토큰 생성	6
0.5.3	4.3 고분산 학습이 만드는 디퓨전 갭	6
0.6	5. BD3-LM을 위한 저분산 노이즈 스케줄	7
0.6.1	5.1 직관: 극단 마스킹률을 피하라	7
0.6.2	5.2 클립 스케줄로 저분산 그래디언트 만들기	7
0.6.3	5.3 블록 크기에 따른 데이터 기반 클립 스케줄	7
0.7	6. 실험	8
0.7.1	6.1 우도 평가	8
0.7.2	6.2 샘플 품질과 가변 길이 시퀀스 생성	9
0.7.3	6.3 절제	9
0.8	7. 논의 및 선행 연구	10
0.9	8. 결론	10
0.10	부록 발췌 (요약)	11

원문: [Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models](#) —
Arriola, Gokaslan, Chiu, Yang, Qi, Han, Sahoo, Kuleshov (Cornell · Cohere · Stanford), ICLR

아래 글은 원문 논문의 순서와 서술을 최대한 따라가며 한국어로 옮긴 것이다. 가능한 한 문장 단위 대응을 유지하되, 한국어 문장만 최소한으로 다듬었다.

0.1 초록

디퓨전 언어모델(diffusion language model)은 병렬 생성과 제어 가능성(controllability)이라는 잠재력 때문에 자기회귀(autoressive, AR) 모델 대비 고유한 이점을 갖지만, 우도 모델링(likelihood modeling)에서 뒤처지며 고정 길이 생성에 간혀 있다. 본 연구에서는 이산 잡음제거 디퓨전(discrete denoising diffusion)과 자기회귀 모델 사이를 보간하는 한 부류의 블록 디퓨전 언어모델(block diffusion language model)을 도입한다. 블록 디퓨전은 가변 길이 생성을 지원하고 KV 캐싱(KV caching)과 병렬 토큰 샘플링으로 추론 효율까지 끌어올림으로써 두 접근법의 핵심 한계를 모두 극복한다. 저자들은 효과적인 블록 디퓨전 모델을 만들기 위한 처방을 제안하는데, 여기에는 효율적인 학습 알고리즘, 그래디언트 분산(variance) 추정량, 그리고 분산을 최소화하는 데이터 기반(data-driven) 노이즈 스케줄(noise schedule)이 포함된다. 블록 디퓨전은 언어모델링 벤치마크에서 디퓨전 모델 가운데 새로운 최첨단(state-of-the-art) 성능을 세우고 임의 길이의 시퀀스 생성을 가능하게 한다. 코드, 모델 가중치, 블로그 글은 프로젝트 페이지에서 제공한다: m-arriola.com/bd3lms.

0.2 1. 서론

디퓨전 모델은 이미지와 비디오 생성에 폭넓게 쓰이며, 텍스트나 생물학적 서열 같은 이산 데이터(discrete data) 생성에도 점점 효과적으로 활용되고 있다. 자기회귀 모델에 비해, 디퓨전 모델은 생성 속도를 가속할 잠재력을 가지며 출력의 제어 가능성을 개선할 수 있다.

이산 디퓨전 모델은 현재 적어도 세 가지 한계에 직면해 있다. 첫째, 챗봇 같은 응용에서는 모델이 임의 길이의 출력 시퀀스(예: 사용자 질문에 대한 답변)를 생성할 수 있어야 하지만, 최근 디퓨전 아키텍처 대부분은 고정 길이 벡터만 생성한다. 둘째, 이산 디퓨전은 생성 시 양방향 맥락(bidirectional context)을 사용하기 때문에 KV 캐싱으로 이전 계산을 재사용할 수 없어 추론 효율이 떨어진다. 셋째, perplexity 같은 표준 지표로 측정한 이산 디퓨전 모델의 품질은 자기회귀 접근에 뒤처져 응용 가능성을 더욱 제약한다.

본 논문은 이러한 한계를 해소하는 방향으로 한 걸음 나아가, 블록 이산 잡음제거 디퓨전 언어모델(Block Discrete Denoising Diffusion Language Models, BD3-LMs)을 도입한다. BD3-LM은 이산 디퓨전과 자기회귀 모델 사이를 보간한다. 구체적으로, 블록 디퓨전 모델(block diffusion model) (반자기회귀(semi-autoregressive) 모델로도 알려져 있음)은 이산 확률변수의 블록(block) 위에서 자기회귀 확률 분포를 정의한다. 이전 블록이 주어졌을 때 한 블록의 조건부 확률은 이산 잡음제거 디퓨전 모델로 명시한다.

효과적인 BD3-LM을 개발하는 데에는 두 가지 도전이 있다. 첫째, 블록 디퓨전 모델의 학습 목적함수를 표준적인 한 번의 신경망 forward pass로 효율적으로 계산하는 것이 불가능해 전용 알고리즘이 필요하다. 둘째, 디퓨전 목적함수의 그래디언트 분산이 커서, 블록 크기 $L' = 1$ 일 때(이때 BD3-LM은 자기회귀 모델과 동가여야 함)조차도 BD3-LM이 자기회귀에 뒤처지게 만든다. 저자들은 그래디언트 분산 추정량을 유도하고, 자기회귀와 디퓨전 사이의 perplexity 갭에 분산이 핵심적으로 기여함을 보인다. 그런 다음 그래디언트 분산을 최소화하는 맞춤형 노이즈 프로세스를 제안하고 perplexity 갭을 줄이는 방향으로 나아간다.

저자들은 BD3-LM을 언어모델링 벤치마크에서 평가하여, 학습 컨텍스트를 초과하는 길이를 포함해 임의

길이의 시퀀스를 생성할 수 있음을 보인다. 또한 BD3-LM은 이산 디퓨전 모델 가운데 새로운 최첨단 perplexity를 달성한다. 임베딩 위에서 가우시안 디퓨전을 수행하는 대안적 반자기회귀 정형(SSD-LM 등)과 비교하여, 본 연구의 이산 접근은 해석 가능한 우도 추정치를 제공하며 더 적은 생성 단계로 더 좋은 생성 perplexity를 달성한다.

요약하면, 본 연구의 기여는 다음과 같다.

- 블록 이산 디퓨전 언어모델을 도입한다. 이 모델은 토큰 블록 위에서 자기회귀이며, 각 블록의 조건부는 이산 디퓨전에 기반한다. 이전 디퓨전 모델과 달리, 블록 디퓨전은 가변 길이 생성과 KV 캐싱을 지원한다.
- 블록 디퓨전 모델을 위한 맞춤형 학습 알고리즘을 도입하여, 모델에 주어진 토큰 배치 전체를 효율적으로 활용할 수 있게 한다.
- 디퓨전 모델 성능의 제약 요인으로 그래디언트 분산을 식별하고, 이를 줄이는 데이터 기반 노이즈 스케줄을 제안한다.
- 결과적으로, 이산 디퓨전의 새로운 최첨단 perplexity를 세우며 자기회귀 모델과의 갭을 줄이는 방향으로 나아간다.

0.3 2. 배경: 언어모델링 패러다임

표기. 저자들은 V 카테고리를 갖는 스칼라 이산 확률변수를 단순체(simplex) Δ^V 안의 원-핫(one-hot) 열벡터 공간 $\mathcal{V} = \{\mathbf{x} \in \{0, 1\}^V : \sum_i x_i = 1\}$ 의 원소로 본다. V 번째 카테고리를 특수한 [MASK] 토큰으로 두며, $\mathbf{m} \in \mathcal{V}$ 가 그 원-핫 벡터다. 길이 L 시퀀스를 $\mathbf{x} = [\mathbf{x}^1, \dots, \mathbf{x}^L]$ 로 표기하며, $\text{Cat}(\cdot; p)$ 는 확률 $p \in \Delta^V$ 의 범주분포(categorical distribution).

0.3.1 2.1 자기회귀 모델

길이 L 토큰 시퀀스 $\mathbf{x}$ 를 데이터 분포 $q(\mathbf{x})$ 로부터 뽑는다. 자기회귀(AR) 모델은 다음과 같은 분해된 분포를 정의한다.

$$\log p_{\theta}(\mathbf{x}) = \sum_{\ell=1}^L \log p_{\theta}(\mathbf{x}^{\ell} \mid \mathbf{x}^{<\ell}),$$

각 $p_{\theta}(\mathbf{x}^{\ell} \mid \mathbf{x}^{<\ell})$ 는 신경망으로 직접 매개변수화된다. AR 모델은 다음 토큰 예측(next token prediction)으로 효율적으로 학습할 수 있지만, 순차 의존성 때문에 L 토큰을 생성하는 데 L 단계가 든다.

0.3.2 2.2 이산 잡음제거 디퓨전 확률 모델 (D3PM)

디퓨전 모델은 클린 데이터 $\mathbf{x}$ 를 점진적으로 노이즈 버전 $\mathbf{x}_t$ ($t \in [0, 1]$)로 만드는 forward 부패 과정(corruption process) q 를 되돌리는 모델 $p_{\theta}(\mathbf{x})$ 를 학습한다. T 단계 이산화에서, $s(j) = (j-1)/T$, $t(j) = j/T$ 로 두고 표기 간략화를 위해 j 를 생략한다 — 일반적으로 s 가 t 직전 시간이다.

D3PM 프레임워크는 q 를 각 토큰 $\mathbf{x}^{\ell}$ 에 독립적으로 작용하는 마르코프 forward 과정으로 정의한다.

$$q(\mathbf{x}_t^{\ell} \mid \mathbf{x}_s^{\ell}) = \text{Cat}(\mathbf{x}_t^{\ell}; Q_t \mathbf{x}_s^{\ell}),$$

여기서 $Q_t \in \mathbb{R}^{V \times V}$ 는 디퓨전 행렬(diffusion matrix)이다. Q_t 는 마스킹, 임의 토큰 변경, 관련 단어 치환 등 다양한 변환을 모델링할 수 있다.

이상적인 디퓨전 모델 p_θ 는 q 의 역방향이다. D3PM은 p_θ 를 다음과 같이 정의한다.

$$p_\theta(\mathbf{x}_s | \mathbf{x}_t) = \prod_{\ell=1}^L p_\theta(\mathbf{x}_s^\ell | \mathbf{x}_t) = \sum_{\mathbf{x}} \left[\prod_{\ell=1}^L q(\mathbf{x}_s^\ell | \mathbf{x}_t^\ell, \mathbf{x}^\ell) p_\theta(\mathbf{x}^\ell | \mathbf{x}_t) \right],$$

잡음제거 베이스 모델 $p_\theta(\mathbf{x}^\ell | \mathbf{x}_t)$ 는 노이즈 시퀀스 $\mathbf{x}_t$ 가 주어졌을 때 클린 토큰 $\mathbf{x}^\ell$ 을 예측한다. 디퓨전 모델 p_θ 는 변분 추론(variational inference) 으로 학습되며, KL 발산을 $\text{KL}[\cdot]$ 로 표기할 때 음의 ELBO(NELBO)는

$$\mathcal{L}(\mathbf{x}; \theta) = \mathbb{E}_q \left[-\log p_\theta(\mathbf{x} | \mathbf{x}_{t(1)}) + \sum_{j=1}^T \text{KL}[q(\mathbf{x}_{s(j)} | \mathbf{x}_{t(j)}, \mathbf{x}) \| p_\theta(\mathbf{x}_{s(j)} | \mathbf{x}_{t(j)})] + \text{KL}[q(\mathbf{x}_{t(T)} | \mathbf{x}) \| p_\theta(\mathbf{x}_{t(T)})] \right].$$

이 정형은 연속 시간 마르코프 체인(CTMC) 이론으로 확장되며, 점수 기반(score-based) 일반화와 ELBO를 더 단단하게 조이는 단순화(MDLM 등)가 알려져 있다.

0.4 3. 블록 디퓨전 언어모델링

저자들은 토큰 블록을 자기회귀로 모델링하면서 각 블록 안에서 디퓨전을 수행해 두 패러다임을 결합한다. 최대 우도 추정(maximum likelihood estimation)을 위한 블록 디퓨전 목적함수와 효율적인 학습·샘플링 알고리즘을 제시한다. 블록 크기가 1일 때 디퓨전 목적함수는 자기회귀 우도와 기댓값상으로 등가임에도 분산이 커서 학습이 약화된다는 점을 보이고, 데이터 기반 노이즈 스케줄로 그래디언트 분산을 줄이는 방안을 제안한다.

0.4.1 3.1 블록 디퓨전 분포와 모델 아키텍처

배경 절의 두 패러다임을 결합하기 위해, $\mathbf{x}$ 의 토큰을 길이 L' 인 $B = L/L'$ 개의 블록으로 묶는다 (B 가 정수라고 가정). 위치 $(b-1)L'$ 부터 bL' 까지의 토큰을 한데 묶은 블록을 $\mathbf{x}^b$ 로 적는다. 우도는 블록에 대해 다음과 같이 분해된다.

$$\log p_\theta(\mathbf{x}) = \sum_{b=1}^B \log p_\theta(\mathbf{x}^b | \mathbf{x}^{<b}),$$

각 $p_\theta(\mathbf{x}^b | \mathbf{x}^{<b})$ 는 길이 L' 블록 위의 이산 디퓨전으로 모델링된다. 식 (디퓨전 정의)에서와 같이, 블록 b 에 한정된 역방향 디퓨전 과정은

$$p_\theta(\mathbf{x}_s^b | \mathbf{x}_t^b, \mathbf{x}^{<b}) = \sum_{\mathbf{x}^b} q(\mathbf{x}_s^b | \mathbf{x}_t^b, \mathbf{x}^b) p_\theta(\mathbf{x}^b | \mathbf{x}_t^b, \mathbf{x}^{<b}).$$

각 항에 NELBO를 적용해 원리적인 학습 목적함수를 얻는다.

$$-\log p_{\theta}(\mathbf{x}) \leq \mathcal{L}_{\text{BD}}(\mathbf{x}; \theta) := \sum_{b=1}^B \mathcal{L}(\mathbf{x}^b, \mathbf{x}^{<b}; \theta).$$

이 합 자체도 유효한 NELBO다.

모델 아키텍처. 결정적으로, B 개의 베이스 잡음제거 모델 $p_{\theta}(\mathbf{x}^b | \mathbf{x}_t^b, \mathbf{x}^{<b})$ 를 단일 신경망 $\mathbf{x}_{\theta}$ 로 매개변수화한다. 이 신경망은 확률뿐 아니라 효율적 학습에 필요한 계산 산출물도 함께 출력하여, $\mathcal{L}_{\text{BD}}(\mathbf{x}; \theta)$ 를 모든 B 블록에 대해 메모리 효율적으로 병렬 계산할 수 있게 한다. 구체적으로 $\mathbf{x}_{\theta}$ 는 블록-인과(block-causal) 어텐션 마스크를 갖는 트랜스포머(Transformer)다. 길이 L 토큰에 적용되며, 블록 b 의 토큰은 블록 1부터 b 까지의 토큰에만 어텐션한다. 학습 중 $\mathbf{x}_{\theta}^b(\mathbf{x}_t^b, \mathbf{x}^{<b})$ 는 노이즈 $\mathbf{x}_t^b$ 와 클린 $\mathbf{x}^{<b}$ 로부터 블록 b 의 잡음제거 토큰 L' 개에 대한 예측을 산출한다.

자기회귀 생성에서 이전 토큰의 키와 값을 캐시해 매 단계 재계산을 피하는 것이 일반적이다. 마찬가지로, 블록 b 의 키-값을 $\mathbf{K}^b, \mathbf{V}^b$ 로 적고 $\mathbf{x}_{\theta}$ 가 이를 입출력으로 받도록 정의한다.

$$\mathbf{x}_{\text{logits}}^b, \mathbf{K}^b, \mathbf{V}^b \leftarrow \mathbf{x}_{\theta}^b(\mathbf{x}_t^b, \mathbf{K}^{1:b-1}, \mathbf{V}^{1:b-1}) := \mathbf{x}_{\theta}^b(\mathbf{x}_t^b, \mathbf{x}^{<b}).$$

0.4.2 3.2 효율적인 학습 및 샘플링 알고리즘

이상적으로는 $\mathcal{L}_{\text{BD}}$ 를 한 번의 forward pass로 계산하고 싶지만, $\mathbf{x}_t^b$ 를 잡음제거하려면 노이즈 입력에 대한 forward가 필요하고 다음 블록을 잡음제거하려면 클린 버전 $\mathbf{x}^b$ 에 대한 forward가 필요하다. 즉 모든 블록은 모델을 적어도 두 번 통과해야 한다.

학습. 이 관찰을 토대로, 최소한의 계산만 요구하는 학습 알고리즘을 제안한다 (Alg. 1). 첫 forward에서 전체 시퀀스 $\mathbf{x}$ 의 키-값 $\mathbf{K}^{1:B}, \mathbf{V}^{1:B}$ 를 미리 계산하고, 이어서 $\mathbf{x}_{\theta}^b(\mathbf{x}_t^b, \mathbf{K}^{1:b-1}, \mathbf{V}^{1:b-1})$ 로 모든 블록의 잡음제거 예측을 계산한다. 각 토큰은 모델을 두 번 통과한다.

벡터화 학습. 단순 구현은 B 번의 루프를 돌지만, 저자들은 클린 데이터 $\mathbf{x}$ 와 노이즈 데이터 $\mathbf{x}_{\text{noisy}} = \mathbf{x}_{t_1}^1 \oplus \dots \oplus \mathbf{x}_{t_B}^B$ 를 이어붙인 입력에 한 번의 forward pass로 $\mathcal{L}_{\text{BD}}$ 를 계산하는 벡터화 구현을 제안한다. $\mathbf{x}_{\text{noisy}} \oplus \mathbf{x}$ 에 대한 어텐션 마스크는, 노이즈 토큰이 자기 블록 안의 다른 노이즈 토큰과 이전 블록의 모든 클린 토큰에만 어텐션하도록 설계된다 (부록 참조). 이로써 BD3-LM 학습 오버헤드가 다룰 만한 수준에 머물고, 사전학습과 결합하면 비용을 더 줄일 수 있다.

샘플링. 한 번에 한 블록씩, 이전에 샘플링된 블록을 조건으로 하여 샘플링한다 (Alg. 2). 임의의 샘플러 $\text{Sample}(\mathbf{x}_{\theta}^b, \mathbf{K}^{1:b-1}, \mathbf{V}^{1:b-1})$ 로 조건부 분포 $p_{\theta}(\mathbf{x}_s^b | \mathbf{x}_t^b, \mathbf{x}^{<b})$ 에서 뽑을 수 있으며, 컨텍스트는 미리 계산된 $\mathbf{K}/\mathbf{V}$ 와의 교차 어텐션으로 주입된다. AR 모델처럼 $\mathbf{K}/\mathbf{V}$ 캐싱이 비용을 절감한다.

특히 본 디퓨전 알고리즘은 임의 길이 시퀀스 샘플링을 가능하게 한다 — 디퓨전 모델이 고정 길이에 갇힌 것과 대비된다. 또한 블록 내부에서는 병렬 생성이 가능해 토큰별 순차 생성에 묶인 AR 샘플러보다 빠르다.

0.5 4. 디퓨전과 AR 모델 사이의 우도 갭 이해

0.5.1 4.1 마스크 BD3-LM

가장 효과적인 디퓨전 LM은 *마스킹(masking)* 노이즈 과정을 활용하며, 토큰을 점진적으로 특수 마스크 토큰으로 치환한다. 저자들은 이 프레임워크에 기반한 한 부류, *마스킹 BD3-LM* 을 도입한다.

토큰별 노이즈 과정은

$$q(\mathbf{x}_t^\ell \mid \mathbf{x}^\ell) = \text{Cat}(\mathbf{x}_t^\ell; \alpha_t \mathbf{x}^\ell + (1 - \alpha_t) \mathbf{m}),$$

여기서 $\mathbf{m}$ 은 마스크 토큰의 원-핫이고 $\alpha_t \in [0, 1]$ 는 t 에 대해 엄격히 감소하며 $\alpha_0 = 1, \alpha_1 = 0$ 인 함수다. 시간 t 의 마스킹 확률은 $1 - \alpha_t$ 다. 단순화된 목적함수(부록 유도 참조)는

$$-\log p_\theta(\mathbf{x}) \leq \mathcal{L}_{\text{BD}}(\mathbf{x}; \theta) := \sum_{b=1}^B \mathbb{E}_{t \sim [0, 1]} \mathbb{E}_q \frac{\alpha'_t}{1 - \alpha_t} \log p_\theta(\mathbf{x}^b \mid \mathbf{x}_t^b, \mathbf{x}^{<b}),$$

α'_t 는 연속시간 극한 ($T \rightarrow \infty$) 에서 α_t 의 순간 변화율. NELBO는 $L' = 1$ 일 때 *타이트(tight)* 하지만 $L' \rightarrow L$ 로 갈수록 진짜 NLL의 느슨한 상계가 된다.

0.5.2 4.2 사례 연구: 단일 토큰 생성

	PPL (↓)
AR	22.88
AR + 무작위 배치 크기	24.37
BD3-LM $L' = 1$	≤ 25.56
BD3-LM $L' = 1$ + 튜닝된 스케줄	22.88

블록 디퓨전 매개변수화는 $L' = 1$ 극한에서 기댓값상 AR NLL과 등가다 (부록 증명). 그런데 LM1B로 학습하면 perplexity가 2점 벌어진다. 이는 높은 학습 분산 때문인데, AR이 L 토큰 전체의 교차 엔트로피로 학습되는 반면 BD3-LM ($L' = 1$) 은 *마스킹된 토큰*에 대해서만 손실을 계산하기 때문 ($\mathbb{E}_t[q(\mathbf{x}_t^\ell = \mathbf{m} \mid \mathbf{x}^\ell)] = 0.5$). 즉 디퓨전 목적함수 학습은 평균적으로 토큰의 절반만으로 그래디언트를 추정한다.

이 갭을 메우기 위해 *항상 모든 토큰을 마스킹* 하는 forward 과정으로 $L' = 1$ BD3-LM 을 학습한다 (즉 $q(\mathbf{x}_t^\ell = \mathbf{m} \mid \mathbf{x}^\ell) = 1$). 이 스케줄에서 디퓨전 목적함수는 AR 목적함수와 *완전히 등가*가 되며, perplexity가 AR 학습과 일치한다 (Table 1). 학습 손실 분산도 줄어든다 — 학습 토큰 328M 시점에 $\text{Var}_{\mathbf{x}, t}[\mathcal{L}_{\text{BD}}(\mathbf{x}; \theta)]$ 가 NELBO 에서 1.52, 완전 마스킹에서 0.11.

0.5.3 4.3 고분산 학습이 만드는 디퓨전 갭

이 관찰을 일반 $L' \geq 1$ 로 확장한다. NELBO는 노이즈 스케줄에 불변이지만, 학습에 쓰는 *몬테카를로 추정량*은 그렇지 않다. 따라서 추정량과 그 그래디언트의 분산이 스케줄에 의존한다. 배치 $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}]$ 에 대한 NELBO 추정량은

$$\mathcal{L}_{\text{BD}}(\mathbf{X}; \theta) = \frac{1}{K} \sum_{k=1}^K \sum_{b=1}^B \frac{\alpha'_{t(k,b)}}{1 - \alpha_{t(k,b)}} \log p_{\theta}(\mathbf{x}^{(k),b} \mid \mathbf{x}_{t(k,b)}^{(k),b}, \mathbf{x}^{(k),<b}),$$

그래디언트 분산은 M 배치 평균으로 추정한다.

$$\text{Var}_{\mathbf{X},t}[\nabla_{\theta} \mathcal{L}] \approx \frac{1}{M-1} \sum_{m=1}^M \left\| \nabla_{\theta} \mathcal{L}(\mathbf{X}^m; \theta) - \frac{1}{M} \sum_{m=1}^M \nabla_{\theta} \mathcal{L}(\mathbf{X}^m; \theta) \right\|_2^2.$$

0.6 5. BD3-LM을 위한 저분산 노이즈 스케줄

0.6.1 5.1 직관: 극단 마스킹률을 피하라

마스킹 세팅에서, 무작위 개수의 토큰을 마스킹하면 모델이 다양한 노이즈 수준을 되돌리는 법을 배운다. 그러나 너무 적게 마스킹하면 복원이 사소해 학습 신호가 빈약하다. 너무 많이 마스킹하면 최적 복원은 데이터 분포의 주변 확률을 외우는 것이며 이 또한 학습에 도움이 안 된다. 두 극단 모두 그래디언트의 분산을 키우므로, 새 스케줄로 클립(*clip*) 하는 법을 배우자는 것이 핵심.

0.6.2 5.2 클립 스케줄로 저분산 그래디언트 만들기

$\beta, \omega \in [0, 1]$ 에 대해 마스킹률을 $1 - \alpha_t \sim \mathcal{U}[\beta, \omega]$ 로 잘라낸 “클립” 노이즈 스케줄 을 제안한다. 몬테카를로 추정의 관점에서 이러한 스케줄은 지정 범위 밖의 마스킹률은 거의 0 또는 거의 1 인 연속 스케줄과 등가다. 이때 범위 안에서는 $\alpha'_t \approx 1/(\beta - \omega)$ 로 선형이다.

0.6.3 5.3 블록 크기에 따른 데이터 기반 클립 스케줄

최적 마스킹률은 블록 크기 L' 에 따라 다를 수 있어, 학습 중 적응적으로 스케줄을 학습한다. [kingma2021variational] 의 분산 최소화 전략은 본 연구의 추정량 (식 분산 추정) 에 직접 적용되지 않는다 — 무작위 t_b 에 더해 무작위 배치 에 걸친 분산까지 줄여야 하기 때문. 대신 학습 분산을 직접 최소화하도록 β, ω 를 최적화한다. 계산 부담을 줄이기 위해 디퓨전 ELBO 추정량의 분산을 그래디언트 분산의 대리(*proxy*) 로 쓴다: $\min_{\beta, \omega} \text{Var}_{\mathbf{X},t}[\mathcal{L}(\mathbf{X}; \theta, \beta, \omega)]$. 학습 중 정기적으로 그리드 서치를 수행한다.

LM1B 결과(Table 2) — NELBO 분산이 테스트 *perplexity* 와 상관 한다. 블록 크기 $L' \in \{4, 16, 128\}$ 마다 NELBO 분산과 *perplexity*를 동시에 최소화하는 고유한 클립 분포가 존재한다.

L'	$\mathcal{U}[0, .5]$	$\mathcal{U}[,3, .8]$	$\mathcal{U}[,5, 1]$	$\mathcal{U}[0, 1]$
128	31.72 / 1.03	31.78 / 1.35	31.92 / 1.83	31.78 / 3.80
16	31.27 / 7.90	31.19 / 3.62	31.29 / 3.63	31.33 / 7.39
4	29.23 / 32.68	29.37 / 10.39	29.16 / 8.28	29.23 / 23.65

(각 셀은 PPL / Var. NELBO. 65B 토큰 학습 후 10B 토큰 파인튜닝.)

0.7 6. 실험

언어모델링 표준 벤치마크에서 BD3-LM을 평가하고, 비조건(unconditional)으로 임의 길이 시퀀스를 생성할 수 있음을 보인다. 베이스 BD3-LM은 최대 블록 크기 $L' = L$ 로 850K 스텝 사전학습한 뒤, One Billion Words(LM1B) 와 OpenWebText(OWT) 에서 다양한 L' 로 150K 스텝 파인튜닝한다.

0.7.1 6.1 우도 평가

LM1B (65B 토큰 학습):

모델	PPL ($\downarrow$)
Transformer (AR)	22.83
Transformer-X Base	23.5
D3PM (absorb)	≤ 82.34
SEDD	≤ 32.68
MDLM	≤ 31.78
BD3-LM $L' = 16$	≤ 30.60
BD3-LM $L' = 8$	≤ 29.83
BD3-LM $L' = 4$	≤ 28.23

MDLM 대비 최대 13% perplexity 개선. OpenWebText에서도 같은 추세.

OWT (524B 토큰 학습):

모델	PPL ($\downarrow$)
AR	17.54
SEDD	≤ 24.10
MDLM	≤ 22.98
BD3-LM $L' = 16$	≤ 22.27
BD3-LM $L' = 8$	≤ 21.68
BD3-LM $L' = 4$	≤ 20.73

제로샷 (OWT 학습 $\rightarrow$ 다른 도메인 평가):

	PTB	Wikitext	LM1B	Lambada	AG News	Pubmed	Arxiv
AR	81.07	25.32	51.14	52.13	52.11	48.59	41.22
SEDD	96.33	35.98	68.14	48.93	67.82	45.39	40.03
MDLM	90.96	33.22	64.94	48.29	62.78	43.13	37.89
BD3-LM $L' = 4$	96.81	31.31	60.88	50.03	61.67	42.52	39.20

Pubmed에서는 AR 도 능가하며, Wikitext·LM1B·AG News·Pubmed 에서 디퓨전 SOTA.

0.7.2 6.2 샘플 품질과 가변 길이 시퀀스 생성

기존 디퓨전 LM의 핵심 단점은 학습 시 정한 출력 컨텍스트 길이를 넘는 시퀀스를 생성할 수 없다는 점이다. OWT는 1024 토큰 학습 컨텍스트보다 긴 문서를 많이 포함해 이 한계를 시험하기에 좋다.

500개 가변 길이 샘플의 길이 통계 (EOS가 나오거나 샘플 엔트로피로 측정한 품질이 크게 떨어질 때까지 생성):

	중앙값 토큰	최대 토큰
OWT 학습셋	717	131K
AR	4008	131K
SEDD	1021	1024
BD3-LM $L' = 16$	798	9982

BD3-LM 은 SEDD 대비 약 10배 긴 시퀀스 생성.

생성 perplexity (GPT2-Large 기준, 110M 파라미터, 524B 토큰 학습):

모델	$L = 1024$ Gen.PPL / NFEs	$L = 2048$ Gen.PPL / NFEs
AR	14.1 / 1K	13.2 / 2K
SEDD	52.0 / 1K	—
MDLM	46.8 / 1K	41.3 / 2K
SSD-LM $L' = 25$	37.2 / 40K	35.3 / 80K
SSD-LM $L' = 25$ (NFE 제한)	281.3 / 1K	281.9 / 2K
BD3-LM $L' = 16$	33.4 / 1K	31.5 / 2K
BD3-LM $L' = 8$	30.4 / 1K	28.2 / 2K
BD3-LM $L' = 4$	25.7 / 1K	23.6 / 2K

이전 디퓨전 방법 모두를 능가하고, SSD-LM 대비 한 자릿수 더 적은 생성 단계로 더 좋은 generative perplexity. 정성적으로도 BD3-LM 샘플은 MDLM보다 일관성이 높고 AR 품질에 근접.

0.7.3 6.3 절제

노이즈 스케줄 (Table 4, LM1B 3B 토큰 파인튜닝):

- $L' = 4$: 클립 $\mathcal{U}[0.45, 0.95]$ 가 PPL 29.21 / 분산 6.24 로 최적 (선형은 30.18 / 23.45).
- $L' = 16$: 클립 $\mathcal{U}[0.3, 0.8]$ 가 PPL 31.12 / 분산 3.58 로 최적 (선형은 31.72 / 7.62).

선형(linear), 코사인(cosine), 로그(logarithmic), 제곱근(square root), 제곱(square) 을 모두 비교했을 때 클립 스케줄이 가장 효과적. 작은 블록일수록 무거운 마스킹, 큰 블록일수록 가벼운 마스킹이 최적이라는 직관이 다른 표준 스케줄과의 비교에서도 확인됨.

학습 알고리즘 효율. 학습 알고리즘에서 logits 계산 옵션은 두 가지 — (1) 두 번 forward (먼저 K/V 미리 계산, 다음에 잡음제거 예측), (2) 두 입력을 같은 어텐션 커널에 이어 붙여 단일 forward. FlashAttention/FlexAttention 같은 효율적 어텐션 커널을 활용해 메모리 대역폭 병목을 줄이면, 단일 forward 가 학습 시간 20~25% 단축. 두 번 forward 비용 대신 더 비싼 어텐션 한 번을 치르는 트레이드오프.

0.8 7. 논의 및 선행 연구

D3PM 과의 비교. 블록 디퓨전은 D3PM 위에 자기회귀 조건부를 얹는다. (1) 고정 시퀀스 길이를 넘어 D3PM 을 확장; (2) D3PM 과 AR 의 perplexity 갭을 분석해 그래디언트 분산을 기여 요인으로 식별하고 분산 최소화 스케줄 설계; (3) D3PM 의 perplexity를 개선. (2)는 본 연구의 블록 확장과 무관하게 *바닐라* D3PM에도 적용 가능. 본 연구는 D3PM의 확장(SED, DiffusionBERT, 연속 시간형 등)에도 적용된다.

MDLM 과의 비교. BD3-LM 은 MDLM의 perplexity 향상 기법을 재사용한다. (1) MDLM은 NELBO가 노이즈 스케줄에 불변이라고 지적했지만, 본 연구는 그래디언트 분산에는 큰 영향이 있음을 보임; (2) MDLM 너머의 perplexity SOTA를 세움. perplexity 개선은 블록 디퓨전뿐 아니라 최적화된 스케줄 에서도 오며, 표준 MDLM과 D3PM 에도 도움이 될 수 있다.

가우시안 디퓨전 과의 비교. 이산 토큰을 연속 임베딩에 매핑한 뒤 가우시안 디퓨전을 적용하는 접근(Diffusion-LM 등)은 연속 데이터용 알고리즘을 활용할 수 있지만 perplexity 가 더 나쁘다.

반자기회귀 디퓨전 과의 비교. SSD-LM(Han et al., 2022; 2023)은 가우시안 기반 블록 형식을 도입했다. BD3-LM 은 D3PM 위에서 (1) 원리적 평가가 가능한 *처리 가능한(tractable)* 우도 추정; (2) 더 빠른 생성 (모델 호출 횟수가 토큰 수로 상한, SSD-LM은 그보다 한 자릿수 많음); (3) 더 좋은 샘플 품질을 갖는다. AR-Diffusion(Wu et al., 2023)은 SSD-LM 을 좌→우 노이즈 스케줄로 확장. PARD(Zhao et al., 2024)는 그래프에 이산 블록 디퓨전을 적용. 반면 본 연구는 (1) AR/디퓨전 성능을 보강; (2) KV 캐싱 지원; (3) 노이즈 블록 내부에서 어텐션 (PARD 는 빈 블록을 새로 주입).

자기회귀 디퓨전 모델(Hoogeboom 2021; Zheng 2024 등)은 임의 순서 자기회귀(*any-order AR*, AO-ARM)의 병렬 샘플링 확장이며, MDLM ↔ AO-ARM 학습이 등가임이 알려져 있다. AR 의 다른 확장으로는 반복 편집(*iterative editing*), 병렬·추측 디코딩(*speculative decoding*), 일관성 학습(*consistency training*), 가이드스(*guidance*), 교차 모달 확장 등이 있다.

한계. BD3-LM 학습은 일반 디퓨전보다 비싸다 (벡터화 알고리즘으로 디퓨전 학습 속도의 <2배 안쪽으로 유지하고, 표준 디퓨전 손실로 사전학습해 갭을 더 줄임). BD3-LM 은 블록을 순차적으로 생성하므로 블록이 작을 때 AR 과 비슷한 속도/제어 가능성 제약을 받을 수 있다. 최적 블록 크기는 과제에 따라 다르다 (예: 더 큰 제어가 필요하면 블록을 키움). 환각(hallucination), 저작권, 제어, 유해 출력 등 생성 모델 일반의 한계는 BD3-LM 에도 그대로 적용된다.

0.9 8. 결론

본 연구는 블록 디퓨전을 탐구하며, 기존 이산 디퓨전의 두 문제 — 임의 길이 생성의 필요성과 자기회귀와의 perplexity 갭 — 에서 출발했다. D3PM 의 블록·단위 확장인 BD3-LM 을 도입하고, 전용 학습 알고리즘과 맞춤형 노이즈 스케줄로 성능을 끌어올렸다. 결과적으로 장문 문서 생성을 가능하게 할 뿐 아니라 perplexity 도 개선해, 이산 디퓨전 모델 중 새로운 최첨단을 달성했다.

0.10 부록 발췌 (요약)

원문 부록은 약 60쪽 분량으로, 다음과 같은 내용을 다룬다.

- 블록 디퓨전 NELBO 유도 — 식의 라인-바이-라인 도출, 마스킹 디퓨전 행렬 Q_t , 시간 t 에서 forward marginal $\bar{Q}_t$, 역방향 사후 $q(\mathbf{x}_s^\ell | \mathbf{x}_t^\ell, \mathbf{x}^\ell)$, 그리고 단순화된 ELBO 의 완전한 도출.
- $L' = 1$ 등가성 증명 — 적절한 스케줄에서 BD3-LM ($L' = 1$) 의 NELBO 가 AR NLL 과 완전히 같음을 증명.
- 벡터화 어텐션 마스크 — 클린/노이즈 토큰을 이어붙인 입력에 대해, 노이즈 토큰이 같은 블록 내 노이즈 토큰 + 이전 블록의 클린 토큰까지만 어텐션하도록 하는 마스크 패턴 시각화.
- FlexAttention 커널 — 단일 forward 알고리즘이 기존 두 forward 대비 20-25% 빠른 이유와 메모리 대역폭 분석.
- 학습 디테일 — LM1B는 1024 컨텍스트 길이, OWT는 1024 컨텍스트 길이; AdamW; 학습률 위밍업; 110M 파라미터 (DiT-style 트랜스포머).
- 추가 절제 — 사전학습-파인튜닝 분할, L' 스케줄 변형, 더 다양한 코퍼스에서의 길이 통계, 생성 샘플 정성 비교.
- 샘플 예시 — AR / MDLM / BD3-LM 의 OWT 샘플 비교; BD3-LM 가 더 주제 일관적인 장문 문서를 생성하는 사례.

자세한 내용은 [arxiv 원문 PDF](#) 를 참고.